appopening

June 22, 2025

[4]:
```
csv_data = """timestamp,whatsapp_opened,snapchat_opened
2025-05-01 08:00,1,0
2025-05-01 13:00,1,1
2025-05-01 18:00,1,
2025-05-01 22:00,,0
2025-05-02 03:00,,
2025-05-02 08:00,,
2025-05-02 14:00,1,0
2025-05-02 18:00,1,
2025-05-02 22:00,,1
2025-05-03 04:00,1,0
2025-05-03 10:00,1,1
2025-05-03 16:00,1,1
2025-05-03 20:00,,0
2025-05-04 02:00,0,1
2025-05-04 06:00,1,1
2025-05-04 11:00,0,0
2025-05-04 16:00,1,0
2025-05-04 21:00,1,
2025-05-05 02:00,,0
2025-05-05 07:00,1,1
2025-05-05 12:00,,1
2025-05-05 16:00,0,1
2025-05-05 21:00,,0
2025-05-06 01:00,0,1
2025-05-06 07:00,1,1
2025-05-06 11:00,,0
2025-05-06 16:00,1,1
2025-05-06 20:00,1,0
2025-05-07 02:00,1,0
2025-05-07 07:00,0,0
2025-05-07 13:00,,
2025-05-07 17:00,0,1
2025-05-07 21:00,,1
2025-05-08 01:00,1,
2025-05-08 05:00,,
2025-05-08 09:00,,1
```

```
2025-05-08 15:00,0,
2025-05-08 19:00,1,
2025-05-09 00:00,0,
2025-05-09 05:00,1,1
2025-05-09 11:00,,0
2025-05-09 15:00,1,0
2025-05-09 19:00,,1
2025-05-09 23:00,1,0
2025-05-10 05:00,,1
2025-05-10 09:00,1,
2025-05-10 13:00,0,0
2025-05-10 18:00,1,0
2025-05-11 00:00,1,1
2025-05-11 05:00,0,1
2025-05-11 09:00,0,1
2025-05-11 13:00,0,0
2025-05-11 18:00,1,1
2025-05-12 00:00,1,0
2025-05-12 04:00,,1
2025-05-12 08:00,1,1
2025-05-12 12:00,,1
2025-05-12 18:00,0,1
2025-05-13 00:00,1,1
2025-05-13 04:00,1,
2025-05-13 09:00,,1
2025-05-13 14:00,0,0
2025-05-13 18:00,1,0
2025-05-14 00:00,1,1
2025-05-14 06:00,,
2025-05-14 11:00,,0
2025-05-14 15:00,1,0
2025-05-14 21:00,1,1
2025-05-15 01:00,,1
2025-05-15 05:00,,0
2025-05-15 11:00,1,
2025-05-15 16:00,,0
2025-05-15 22:00,1,
2025-05-16 02:00,,1
2025-05-16 07:00,,0
2025-05-16 11:00,,0
2025-05-16 16:00,1,
2025-05-16 22:00,1,1
2025-05-17 02:00,0,
2025-05-17 06:00,1,
2025-05-17 10:00,1,
2025-05-17 15:00,,0
2025-05-17 19:00,1,0
```

```
2025-05-18 01:00,0,1
2025-05-18 06:00,1,
2025-05-18 10:00,1,
2025-05-18 14:00,,1
2025-05-18 20:00,1,1
2025-05-19 00:00,1,0
2025-05-19 05:00,,0
2025-05-19 10:00,1,
2025-05-19 15:00,1,
2025-05-19 19:00,1,1
2025-05-20 00:00,1,
2025-05-20 04:00,0,1
2025-05-20 08:00,0,
2025-05-20 13:00,1,
2025-05-20 18:00,0,1
2025-05-20 22:00,0,1
"""
```

[3]:
```python
import pandas as pd
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import matplotlib.pyplot as plt
from io import StringIO # Keep StringIO for embedding data directly

# --- Simplified Workflow: NLP & Forecasting ---
# This workflow outlines essential steps for building predictive models.
# It's applicable for both text-based (NLP) and time-based (Forecasting) data.

# --- 1. Prepare Your Data ---
print("--- 1. Prepare Your Data ---")
df = pd.read_csv(StringIO(csv_data))

df['whatsapp_opened'] = df['whatsapp_opened'].fillna(0).astype(int)
df['snapchat_opened'] = df['snapchat_opened'].fillna(0).astype(int)
df['timestamp'] = pd.to_datetime(df['timestamp'])
df.set_index('timestamp', inplace=True)
df['total_app_opened'] = df['whatsapp_opened'] + df['snapchat_opened']
daily_data = df['total_app_opened'].resample('D').sum().reset_index()
daily_data.rename(columns={'timestamp': 'Date', 'total_app_opened': 'Value'},␣
 ↪inplace=True)

n_lags = 3 # Use last 3 days for prediction
```

```
def create_sequences(data, n_lags):
    X, y = [], []
    for i in range(len(data) - n_lags):
        X.append(data[i:(i + n_lags)])
        y.append(data[i + n_lags])
    return np.array(X), np.array(y)

scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(daily_data['Value'].values.reshape(-1, 1))

X, y = create_sequences(scaled_data, n_lags)
X = X.reshape(X.shape[0], X.shape[1], 1)

test_size_days = 3
train_size = len(daily_data) - test_size_days - n_lags

X_train, X_test = X[0:train_size,:], X[train_size:len(X),:]
y_train, y_test = y[0:train_size], y[train_size:len(y)]

print(f"Data points: {len(daily_data)} | Train: {len(X_train)} | Test:␣
 ↪{len(X_test)}")


# --- 2. Build Your Model ---
print("\n--- 2. Build Your Model ---")
model = Sequential()
model.add(LSTM(50, activation='relu', input_shape=(n_lags, 1)))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mean_squared_error')
print(model.summary())


# --- 3. Train Your Model ---
print("\n--- 3. Train Your Model ---")
print("Training model (epochs=100)...")
history = model.fit(X_train, y_train, epochs=100, batch_size=1, verbose=0)
print("Training complete.")


# --- 4. Evaluate & Refine ---
print("\n--- 4. Evaluate & Refine ---")
y_pred_scaled = model.predict(X_test, verbose=0)
y_pred = scaler.inverse_transform(y_pred_scaled)
y_test_original = scaler.inverse_transform(y_test)

rmse = np.sqrt(mean_squared_error(y_test_original, y_pred))
mae = mean_absolute_error(y_test_original, y_pred)
```

```
r2 = r2_score(y_test_original, y_pred)

print(f"Test RMSE: {rmse:.2f} | MAE: {mae:.2f} | R2: {r2:.2f}")

plt.figure(figsize=(10, 6))
plt.plot(daily_data['Date'].iloc[len(daily_data) - len(y_test_original):],␣
 ↪y_test_original, label='Actual', marker='o')
plt.plot(daily_data['Date'].iloc[len(daily_data) - len(y_pred):], y_pred,␣
 ↪label='Predicted', marker='x')
plt.title('Actual vs. Predicted (Test Set)')
plt.xlabel('Date')
plt.ylabel('Total App Openings')
plt.xticks(rotation=45)
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig('test_predictions_plot.png')
print("Generated 'test_predictions_plot.png'.")


# --- 5. Make Predictions ---
print("\n--- 5. Make Predictions ---")
last_known_data = scaled_data[-n_lags:].reshape(1, n_lags, 1)
future_predictions_scaled = []
forecast_dates = pd.date_range(start=daily_data['Date'].iloc[-1] + pd.
 ↪Timedelta(days=1), periods=11)

for _ in range(11):
    next_pred_scaled = model.predict(last_known_data, verbose=0)[0, 0]
    future_predictions_scaled.append(next_pred_scaled)
    # Fix: Reshape next_pred_scaled to 3D to match last_known_data dimensions␣
 ↪for concatenation
    last_known_data = np.append(last_known_data[:, 1:, :],␣
 ↪[[[next_pred_scaled]]], axis=1)

future_predictions_original = scaler.inverse_transform(np.
 ↪array(future_predictions_scaled).reshape(-1, 1))

print("\n--- 11-Day Forecast ---")
forecast_df = pd.DataFrame({'Date': forecast_dates, 'Predicted_App_Openings':␣
 ↪future_predictions_original.flatten().round(2)})
print(forecast_df.to_string(index=False))

plt.figure(figsize=(12, 7))
plt.plot(daily_data['Date'], daily_data['Value'], label='Historical',␣
 ↪marker='o', markersize=4)
```

```
plt.plot(forecast_df['Date'], forecast_df['Predicted_App_Openings'],␣
  ↪label='Forecast', linestyle='--', marker='x', color='red')
plt.title('Historical and Forecasted App Openings')
plt.xlabel('Date')
plt.ylabel('Total App Openings')
plt.xticks(rotation=45)
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig('full_forecast_plot.png')
print("Generated 'full_forecast_plot.png'.")
```

```
--- 1. Prepare Your Data ---
Data points: 20 | Train: 14 | Test: 3

--- 2. Build Your Model ---

C:\Users\bhara\anaconda3\Lib\site-packages\keras\src\layers\rnn\rnn.py:199:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(**kwargs)
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
lstm_1 (LSTM)	(None, 50)	10,400
dense_1 (Dense)	(None, 1)	51

Total params: 10,451 (40.82 KB)

Trainable params: 10,451 (40.82 KB)

Non-trainable params: 0 (0.00 B)

```
None

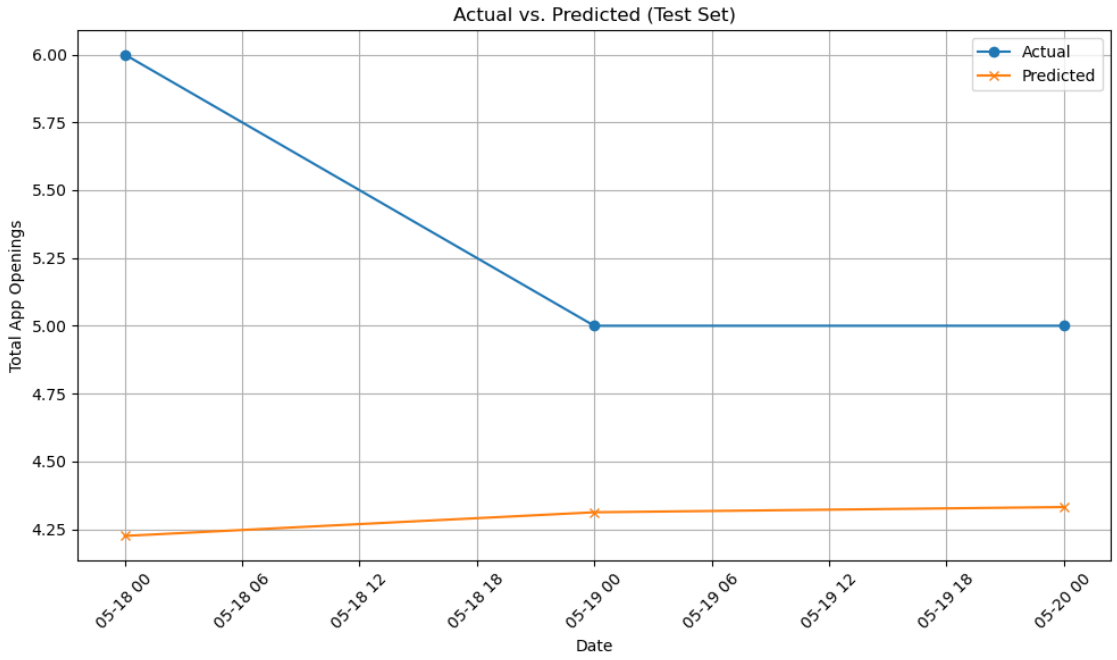
--- 3. Train Your Model ---
Training model (epochs=100)…
```

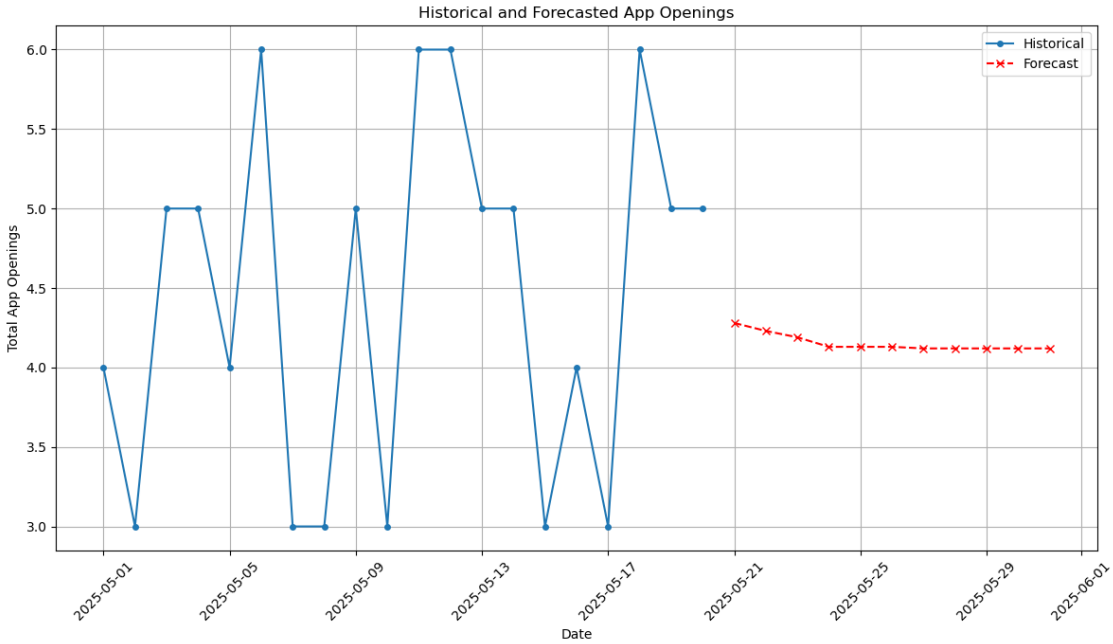
```
Training complete.

--- 4. Evaluate & Refine ---
Test RMSE: 1.16 | MAE: 1.04 | R2: -5.10
Generated 'test_predictions_plot.png'.

--- 5. Make Predictions ---

--- 11-Day Forecast ---
      Date  Predicted_App_Openings
2025-05-21                    4.28
2025-05-22                    4.23
2025-05-23                    4.19
2025-05-24                    4.13
2025-05-25                    4.13
2025-05-26                    4.13
2025-05-27                    4.12
2025-05-28                    4.12
2025-05-29                    4.12
2025-05-30                    4.12
2025-05-31                    4.12
Generated 'full_forecast_plot.png'.
```




Historical and Forecasted App Openings
Historical
Forecast
Total App Openings
6.0
5.5
5.0
4.5
4.0
3.5
3.0
2025-05-01
2025-05-05
2025-05-09
2025-05-13
2025-05-17
2025-05-21
2025-05-25
2025-05-29
2025-06-01
Date

[ ]: